



CarRent + Spring Boot

Integracja projektu

Zmiany

Zmieniamy sposób uruchamiania i komunikacji z projektem, nie sens działania aplikacji.

UIConsole

Projekt CarRent działał jako aplikacja konsolowa. Main ręcznie tworzył repozytoria, serwisy i UI, a użytkownik wybierał opcje w terminalu.

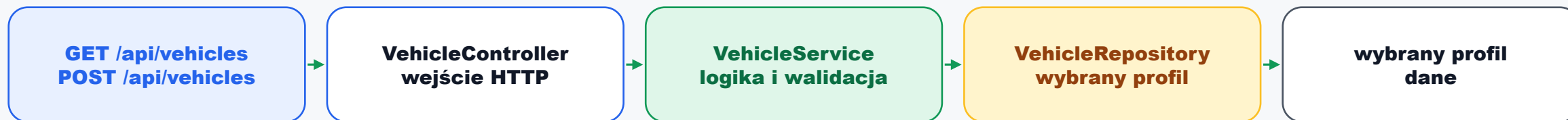


Spring

Ten sam projekt ma działać jako aplikacja webowa: Posiada endpointy REST, JSON w odpowiedziach i beany tworzone przez kontener Springa.



Vehicle



Analogiczne elementy

- UserController działa analogicznie
- RentalController wywołuje RentalService
- CategoryController obsługuje kategorie
- adaptery repozytoriów powtarzają ten sam pomysł
- szczegóły zapisu danych zależą od profilu

Przed Springiem: ręczne składanie aplikacji

Main decydował, które obiekty powstaną i jak zostaną połączone.

```
String storageType = args.length > 0 ? args[0] : "json";

VehicleRepository vehicleRepository = new VehicleJsonRepository();
RentalRepository rentalRepository = new RentalJsonRepository();

VehicleService vehicleService = new VehicleService(
    vehicleRepository,
    rentalRepository,
    vehicleValidator
);

UI ui = new UI(authService, vehicleService, rentalService,
userService, categoryService);
ui.start();
```

Co to oznaczało?

Gdy chcieliśmy zmienić sposób przechowywania danych, zmieniał się kod startowy.

Brak aplikacji webowej

Aplikacja nie miała wejścia HTTP. UI było terminalowe, więc Postman, przeglądarka albo frontend nie mogły bezpośrednio wykonywać operacji.

Większość aplikacji jest gotowa

Mamy logikę serwisów i interfejsy repozytoriów. Nie trzeba przepisywać wszystkiego od zera.

Po integracji: Spring Boot uruchamia aplikację

Nowy punkt startowy jest krótki, bo resztę konfiguracji przejmują Spring.

```
package com.umcsuser.carrent;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CarRentApplication {

    public static void main(String[] args) {
        SpringApplication.run(CarRentApplication.class, args);
    }
}
```

```
import org.springframework.stereotype.Component;

@Component
public class VehicleValidator {
    //...
```

SpringApplication
n.run()

component scan

beany

server HTTP
localhost:8080

@SpringBootApplication robi trzy rzeczy: włącza autokonfigurację, skanuje komponenty i oznacza klasę jako główną konfigurację aplikacji.

W praktyce: Spring znajduje klasy z @Component, @RestController, @Service, @Repository oraz połączy je przez konstruktory.

UWAGA! Walidator, też jest komponentem Springa!!

Krok 1: POM zamieniamy na Spring Boot

Startery dają gotowe zestawy zależności zamiast ręcznego dobierania każdego modułu.

Stary POM: wersje i biblioteki ręcznie

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>

<dependency>
  <artifactId>hibernate-core</artifactId>
  <version>6.4.4.Final</version>
</dependency>
<dependency>
  <artifactId>jackson-databind</artifactId>
  <version>2.18.3</version>
</dependency>
```

Nowy POM: parent + startery

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.4</version>
</parent>

<dependency>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

Wersje

Spring Boot starter-parent zarządza wersjami zależności. Startery dostarczają Web, JSON, JPA, JDBC jako gotowe pakiety.

Krok 2: profile Springa wybierają wariant aplikacji

Ten sam kod uruchamiamy jako JSON, JDBC albo JPA przez APP_PROFILE.

application.yml – profil aktywny

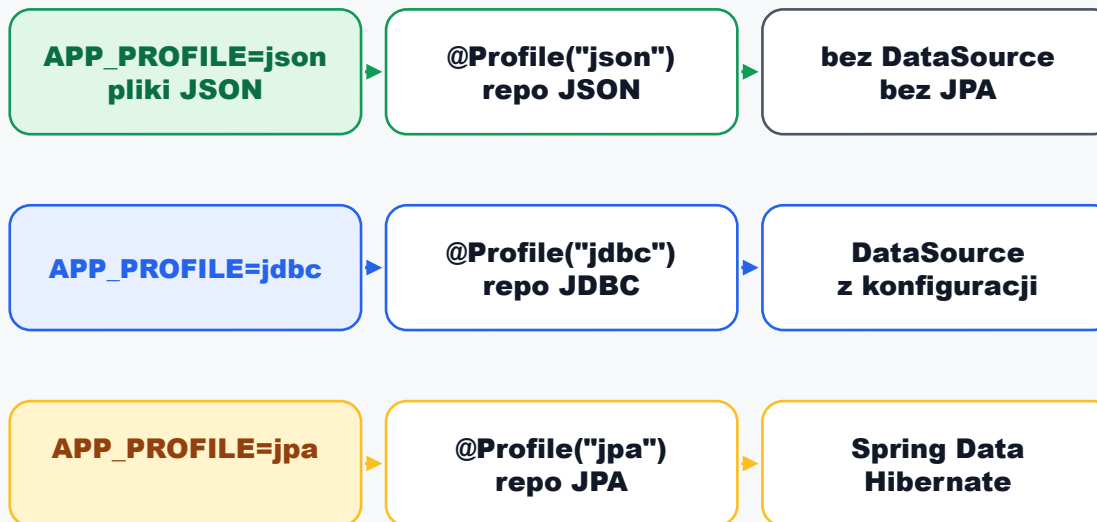
```
spring:
  profiles:
    active: ${APP_PROFILE:json}
  main:
    banner-mode: off

server:
  port: 8080

current:
  json:
    users-file: users.json
    vehicles-file: vehicles.json
    rentals-file: rentals.json
    categories-file: categories.json
```

Brak APP_PROFILE oznacza json, więc aplikacja startuje bez bazy danych.

Co oznacza aktywny profil?



Dwie role profilu

Profil wybiera beany, np. które repozytorium implementuje VehicleRepository. Plik application-*.yml dokłada konfigurację środowiska, np. DataSource albo wyłączenia autokonfiguracji.

Krok 2a: exclude w profilach

Startery JPA/JDBC są w projekcie cały czas, ale nie każdy profil powinien uruchamiać całą infrastrukturę.

application-json.yml

```
spring:
  autoconfigure:
    exclude:
      - DataSourceAutoConfiguration
      - HibernateJpaAutoConfiguration
      - JpaRepositoriesAutoConfiguration
```

application-jdbc.yml

```
spring:
  autoconfigure:
    exclude:
      - HibernateJpaAutoConfiguration
      - JpaRepositoriesAutoConfiguration
  datasource:
    url: ${DB_URL}
    driver-class-name: org.postgresql.Driver
```

Profil json

Wyłącza DataSource i Hibernate/JPA, bo dane czytamy z plików. Gdyby tego nie było, Spring mógłby próbować utworzyć połączenie z bazą mimo pracy na JSON.

Profil jdbc

Wyłącza tylko JPA, ale zostawia DataSource. Repozytorium JDBC nadal potrzebuje połączenia, tylko dostaje je od Springa.

Profil jpa

Nie wyłącza JPA. Włącza konfigurację spring.datasource oraz spring.jpa, bo Spring Data JPA i Hibernate są tutaj właściwym mechanizmem dostępu do danych.

W pliku YAML używamy pełnych nazw klas - Na slajdzie są skrócone do czytelności.

- org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
- org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration
- org.springframework.boot.autoconfigure.data.jpa.JpaRepositoriesAutoConfiguration

Krok 2b: jpa

Uruchomiamy jeden profil.

Profil JPA – przykładowa konfiguracja

```
spring:
  datasource:
    url: ${DB_URL}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

Ustawienia zmiennych

APP_PROFILE=jpa oraz DB_URL wskazujący na naszą bazę. To wystarczy do uruchomienia wariantu JPA.

Repo

Repo ma kilka profili, ale uruchamiamy jeden naraz. Mechanizm Springa pozostaje taki sam.

Krok 2c: przełączanie profili w praktyce

Zmiana profilu to zmiana sposobu działania tej samej aplikacji, bez przepisywania serwisów i kontrolerów.

Wariant	Aktywne profile	Efekt
JSON	APP_PROFILE=json albo brak zmiennej	repozytoria JSON, brak bazy, brak DataSource
JDBC	APP_PROFILE=jdbc + DB_URL	repozytoria JDBC, DataSource od Springa, JPA wyłączone
JPA	APP_PROFILE=jpa + DB_URL	repozytoria Spring Data JPA, Hibernate, transakcje

Krok 3: repozytorium JSON staje się beanem

Najprostsza zmiana: ta sama implementacja, ale zarządzana przez Springa.

Przed

```
public class VehicleJsonRepository
    implements VehicleRepository {

    private final JsonFileStorage<Vehicle> storage =
        new JsonFileStorage<>("vehicles.json", ...);

    public VehicleJsonRepository() {
        this.vehicles = new ArrayList<>(storage.load());
    }
}
```

Po

```
@Repository
@Profile("json")
public class VehicleJsonRepository
    implements VehicleRepository {

    public VehicleJsonRepository(
        @Value("${carrent.json.vehicles-file}") String filename) {
        this.storage = new JsonFileStorage<>(filename, ...);
        this.vehicles = new ArrayList<>(storage.load());
    }
}
```

@Repository

@Profile("json")

@Value("\${carrent.json.vehicles-file}") String filename)

Nie tworzymy repozytorium przez new. Spring tworzy bean tylko wtedy, gdy aktywny jest profil json. Nazwa pliku nie jest już zaszyta w kodzie, tylko pochodzi z application.yml.

Krok 4: repozytorium JDBC używa DataSource

DataSource ze Springa

```
@Repository
@Profile("jdbc")
public class VehicleJdbcRepository
    implements VehicleRepository {

    private final DataSource dataSource;

    public VehicleJdbcRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public List<Vehicle> findAll() {
        try (Connection connection = dataSource.getConnection()) {
            // SQL...
        }
    }
}
```

DataSource w frameworku Spring to komponent, który reprezentuje fabrykę połączeń z bazą danych. Jest to fundamentalny element pozwalający aplikacji Spring na komunikację z relacyjną bazą danych (SQL) poprzez JDBC (Java Database Connectivity).

Krok 5: adapter JPA bez zmiany interfejsu

Spring Data JPA może pracować pod spodem, a serwisy nadal używają VehicleRepository.

```
@Profile("jpa")
public interface VehicleJpaRepository
    extends JpaRepository<Vehicle, String> {
}
```

```
@Repository
@Profile("jpa")
public class VehicleRepositoryJpaAdapter
    implements VehicleRepository {

    private final VehicleJpaRepository delegate;

    public List<Vehicle> findAll() {
        return delegate.findAll();
    }

    public Vehicle save(Vehicle vehicle) {
        if (vehicle.getId() == null) {
            vehicle.setId(UUID.randomUUID().toString());
        }
        return delegate.save(vehicle);
    }
}
```



Adapter

Serwis nie musi wiedzieć, czy dane są w JSON, JDBC czy JPA. Zależność serwisu zostaje na starym interfejsie VehicleRepository.

Logika

Nie przepisujemy logiki. Zmieniamy implementację danych, a kod serwisu nadal pracuje na tym samym interfejsie.

Krok 6: serwis zostaje, ale Spring nim zarządza

Business logic zostaje w serwisie. Dodajemy rolę beana i granice transakcji.

Przed: zwykła klasa

```
public class VehicleService
    implements VehicleServiceInterface {

    private final VehicleRepository vehicleRepo;
    private final RentalRepository rentalRepo;

    public List<Vehicle> findAvailableVehicles() {
        return vehicleRepo.findAll().stream()
            .filter(v -> rentalRepo
                .findByVehicleIdAndReturnDateIsNull(v.getId())
                .isEmpty())
            .toList();
    }
}
```

Po: bean + transakcje

```
@Service
@Transactional
public class VehicleService
    implements VehicleServiceInterface {

    private final VehicleRepository vehicleRepo;
    private final RentalRepository rentalRepo;

    @Transactional(readOnly = true)
    public List<Vehicle> findAvailableVehicles() {
        return vehicleRepo.findAll().stream()
            .filter(v -> rentalRepo
                .findByVehicleIdAndReturnDateIsNull(v.getId())
                .isEmpty())
            .toList();
    }
}
```

Zmiana – przez adnotacje!

Kod logiki prawie się nie zmienia. Różnica polega na tym, że Spring może utworzyć serwis, wstrzyknąć repozytoria i objąć metody transakcją.

Krok 7: Controller jako nowe wejście do aplikacji

Endpoint REST mapuje adres URL na metodę Javy i deleguje do serwisu.

```
@RestController
@RequestMapping("/api/vehicles")
public class VehicleController {

    private final VehicleServiceInterface vehicleService;

    @GetMapping
    public List<Vehicle> list(
        @RequestParam(name = "available", defaultValue = "false") boolean
        available) {
        return available ? vehicleService.findAvailableVehicles()
            : vehicleService.findAllVehicles();
    }

    @PostMapping
    public Vehicle create(@RequestBody Vehicle vehicle) {
        return vehicleService.addVehicle(vehicle);
    }
}
```

Metoda	URL	Co robi
GET	/api/vehicles	lista pojazdów
GET	/api/vehicles?available=true	tylko dostępne
GET	/api/vehicles/{id}	jeden pojazd
POST	/api/vehicles	dodanie pojazdu
DELETE	/api/vehicles/{id}	usunięcie

VehicleController — pełny kod 1/2

Pakiet, importy, adnotacje i wstrzyknięcie serwisu

**@RestController + @RequestMapping =
adresy HTTP zwracające JSON**

```
package com.umcsuser.carrent.web;

import com.umcsuser.carrent.models.Vehicle;
import com.umcsuser.carrent.services.VehicleServiceInterface;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/vehicles")
public class VehicleController {

    private final VehicleServiceInterface vehicleService;

    public VehicleController(VehicleServiceInterface vehicleService) {
        this.vehicleService = vehicleService;
    }
}
```


VehicleController — pełny kod 2/2

Endpointy dla listy, szczegółów, tworzenia i usuwania pojazdu

Controller deleguje pracę do VehicleService — nie zawiera SQL ani logiki profili

```
@GetMapping
public List<Vehicle> list(
    @RequestParam(name = "available", required = false, defaultValue = "false")
    boolean available
) {
    return available
        ? vehicleService.findAvailableVehicles()
        : vehicleService.findAllVehicles();
}

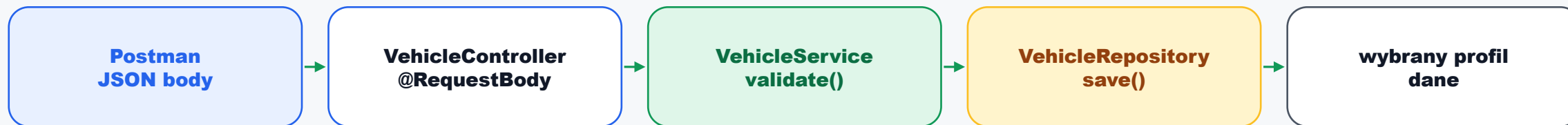
@GetMapping("/{id}")
public Vehicle get(@PathVariable String id) {
    return vehicleService.findById(id);
}

@PostMapping
public Vehicle create(@RequestBody Vehicle vehicle) {
    return vehicleService.addVehicle(vehicle);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable String id) {
    vehicleService.removeVehicle(id);
    return ResponseEntity.noContent().build();
}
```

Warstwy po integracji

Kontroler obsługuje HTTP, serwis logikę, repozytorium dane.



Controller

Czyta parametry z URL i body żądania. Nie decyduje, czy pojazd jest poprawny.

Service

Zna reguły: walidacja kategorii, sprawdzenie aktywnego wypożyczenia, wyjątki biznesowe.

Repository

Ukrywa szczegóły zapisu: pliki JSON, JDBC albo JPA. Serwis zależy od interfejsu.

Krok 8: wyjątki biznesowe zamieniamy na HTTP

ControllerAdvice robi mapowanie wyjątku z serwisu.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, String>>
    badRequest(IllegalArgumentException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("error", e.getMessage()));
    }

    @ExceptionHandler(IllegalStateException.class)
    public ResponseEntity<Map<String, String>> conflict(IllegalStateException
e) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(Map.of("error", e.getMessage()));
    }
}
```

400 Bad Request

Dane wejściowe są niepoprawne: brak kategorii, błędny atrybut, nieznany pojazd.

409 Conflict

Żądanie jest logicznie sprzeczne ze stanem systemu: ktoś już wypożyczył pojazd albo użytkownik ma aktywne wypożyczenie.

Handler

Wyjątki z serwisu nie są tylko problemem Javy. W aplikacji webowej muszą stać się czytelną odpowiedzią HTTP.

Testujemy API w Postmanie

Jeden zestaw żądań wystarczy, żeby sprawdzić cały przepływ Vehicle.

#	Żądanie	Cel
1	GET http://localhost:8080/api/vehicles	lista
2	GET /api/vehicles?available=true	dostępne
3	POST /api/vehicles	dodanie
4	DELETE /api/vehicles/{id}	usunięcie

Body dla POST

```
{
  "category": "Car",
  "brand": "Toyota",
  "model": "Corolla",
  "year": 2020,
  "plate": "LU12345",
  "price": 120.0,
  "attributes": {
    "fuelType": "petrol"
  }
}
```

Kategoria

Kategoria musi pasować do categories.json. Dla przykładu z repo jest to „Car”, a wymagany atrybut to string fuelType.

Porównanie kodu 1: uruchomienie

Największa różnica: stary projekt sam składał obiekty, nowy oddaje to Springowi.

Stary kod: Main.java

```
public static void main(String[] args) {
    String storageType = args.length > 0
        ? args[0].toLowerCase()
        : "json";

    VehicleRepository vehicleRepository =
        new VehicleJsonRepository();
    RentalRepository rentalRepository =
        new RentalJsonRepository();

    VehicleService vehicleService =
        new VehicleService(vehicleRepository, rentalRepository,
            validator);

    UI ui = new UI(...);
    ui.start();
}
```

Nowy kod: CarRentApplication.java

```
@SpringBootApplication
public class CarRentApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            CarRentApplication.class,
            args
        );
    }
}
```

Porównanie kodu 2: repozytorium JDBC

Zmiana sposobu dostępu do połączenia.

Stary kod: singleton

```
public class VehicleJdbcRepository
    implements VehicleRepository {

    public List<Vehicle> findAll() {
        try (Connection connection =
            JdbcConnectionManager
                .getInstance()
                .getConnection()) {
            // SQL...
        }
    }
}
```

Nowy kod: DataSource z Springa

```
@Repository
@Profile("jdbc")
public class VehicleJdbcRepository
    implements VehicleRepository {

    private final DataSource dataSource;

    public VehicleJdbcRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public List<Vehicle> findAll() {
        try (Connection connection = dataSource.getConnection()) {
            // SQL...
        }
    }
}
```

Porównanie kodu 3: ręczny Hibernate → Spring Data JPA

Ręczny SessionFactory znika, a repozytorium JPA tworzy Spring.

Stary wariant Hibernate

```
Configuration configuration = new Configuration();
configuration.setProperty("hibernate.connection.url",
    System.getenv("DB_URL"));
configuration.setProperty("hibernate.hbm2ddl.auto", "update");

configuration.addAnnotatedClass(User.class);
configuration.addAnnotatedClass(Vehicle.class);
configuration.addAnnotatedClass(Rental.class);

SessionFactory sessionFactory = configuration.buildSessionFactory();
```

Nowy wariant JPA

```
@Profile("jpa")
public interface RentalJpaRepository
    extends JpaRepository<Rental, String> {

    Optional<Rental>
        findByVehicle_IdAndReturnDateTimeIsNull(String vehicleId);
}

@Repository
@Profile("jpa")
class RentalRepositoryJpaAdapter implements RentalRepository { ... }
```

Hibernate

Hibernate nadal działa pod spodem, ale Spring Boot konfiguruje go automatycznie na podstawie starterów i application.yml.

User, Rental i Category

Pozostałe kontrolery realizują ten sam wzorzec na innych zasobach.

Obszar	Kontroler	Przykładowe endpointy
Pojazdy	VehicleController	GET /api/vehicles, POST /api/vehicles
Użytkownicy	UserController	GET /api/users, GET /api/users/{id}
Wypożyczenia	RentalController	GET /api/rentals, POST /api/rentals/users/{userId}/rent/{vehicleId}
Kategorie	CategoryController	GET /api/categories, GET /api/categories/{category}

Najczęstsze problemy przy uruchamianiu

Te błędy pojawiają się najczęściej po przejściu na Spring Boot.

Zły profil

Aplikacja startuje na json, bo APP_PROFILE nie jest ustawione. Dla bazy trzeba podać APP_PROFILE=jpa albo APP_PROFILE=jdbc.

Brak DB_URL

Profil jpa/jdbc wymaga zmiennej DB_URL. Bez niej Spring nie zbuduje DataSource.

Dwa beany jednego interfejsu

Jeżeli implementacje repozytoriów nie mają @Profile, Spring nie wie, którą wstrzyknąć.

Niepoprawny JSON

POST musi mieć Content-Type: application/json, a category i attributes muszą pasować do categories.json.

Podsumowanie

Integracja ze Springiem to przede wszystkim zmiana sposobu składania i udostępniania aplikacji.

Najważniejsze wnioski

- Nie przepisujemy całej aplikacji: serwisy i interfejsy zostają.
- Main/UI zastępują CarRentApplication i kontrolery REST.
- Spring tworzy obiekty i wstrzykuje zależności przez konstruktory.
- Profile wybierają implementację repozytorium.

Kolejny krok: uruchomienie aplikacji i test endpointów w Postmanie

Zadanie

Praktyczny checklist integracji aplikacji ze Spring Boot.

- 1 POM** dodać Spring Boot parent, starter-web, starter-data-jpa/jdbc, validation, plugin
- 2 Start** dodać CarRentApplication z @SpringBootApplication
- 3 Beany** oznaczyć serwisy @Service, repozytoria @Repository, komponenty @Component, zależności przez konstruktor
- 4 Profile** dodać profile @Profile i application.yml z APP_PROFILE, plus oddzielne ustawienia- jpa,jdbc,json
- 5 Repo/Service** w JPA dodać JpaRepository + adapter do starego interfejsu, przerobić pozostałe repozytoria wykorzystując Springa
Użyć jednej wersji serwisów z odpowiednimi adnotacjami
- 6 Web** dodać kontrolery REST i mapowania /api/... Rental można zrobić po idkach w @PathVariable. Bez loginu i rejestracji Userów
- 7 Błędy** dodać ControllerAdvice dla 400/409
- 8 Test** uruchomić profil, sprawdzić endpointy w Postmanie

Dziękuję za uwagę

CarRent + Spring Boot

